CUDA-Accelerated FP-Growth Support-Threshold Study

Ten datasets, forty support settings, and CPU/CUDA performance analysis

Abstract

This project implements complete frequent-itemset mining using FP-Growth: a compressed frequent-pattern tree, linked headers, conditional pattern bases and recursive conditional trees. A separate CUDA build accelerates weighted support-counting histograms for the initial database and every recursive conditional tree. Tree construction and recursion remain on the CPU, making this a genuine **hybrid CUDA** implementation, not a fully GPU-resident tree miner.

Experiments evaluate ten distinct transaction datasets at four minimum-support thresholds each. Per-threshold measurements include exact frequent-itemset count, CPU time, host resident memory, and - where actual NVIDIA measurements exist - CUDA time, CUDA memory, kernel counts and CPU/CUDA speedup. Each dataset has its own four-panel performance figure and a results table.

Dataset provenance: four UCI-derived datasets and six reproducibly generated synthetic workloads. These are not ten UCI datasets. All measurements are from this project; none were imported from the comparison project.

Measured settings	Benchmark status	GPU status
10 x 4 = 40	CPU experiments completed	Real GPU sweep pending

Measured environment: AMD EPYC 9V74 80-Core Processor; Linux-6.18.55-x86_64-with-glibc2.41; GPU: not benchmarked in these records.

Timing: median of 2 independent runs per support level; C++ process timing including input parsing and mining, excluding Python startup and pattern CSV writing. Very small runs are sensitive to timer noise.

1. Problem definition and FP-Growth method

Given a transaction database D with N rows and an itemset X, support(X) is the number of transactions containing every item of X. At minimum support s, X is frequent if support(X) is at least ceil(sN). Our miner outputs every nonempty frequent itemset and its support, with duplicates removed inside each transaction.

1.1 Sequential miner

The initial scan counts item supports and discards infrequent tokens. Remaining items are sorted by decreasing support and inserted into an FP-tree. Shared prefixes share nodes. Header links connect nodes with the same item. For each suffix item, node-to-root ancestor paths form a weighted conditional pattern base. A conditional FP-tree is then built and recursively mined.

1.2 CUDA hybrid: actual GPU responsibilities

For both the root tree and recursively generated conditional pattern bases, CPU code flattens item IDs and path multiplicities into arrays. These arrays are copied to the NVIDIA device. A 64-bit atomicAdd histogram kernel computes weighted support values in parallel. The histogram returns to the CPU, which prunes and rebuilds the tree. This can accelerate large counting workloads but may be slower for many tiny conditional trees because of kernel-launch and transfer overhead.

```
CPU: generate weighted transactions from conditional pattern base
  -> flatten [item_id], [weight]
GPU: transfer arrays, reset support histogram, launch atomicAdd kernel
  -> transfer support histogram back to host
CPU: remove infrequent items, build conditional tree, recurse
```

1.3 Correctness approach

The weighted support of item j in a conditional base is the sum of path weights of all rows containing j. Each suffix ordering is explored once, so every frequent itemset is emitted once. The same input and threshold are evaluated with the CPU and actual CUDA executables. Their complete itemset/support CSVs are compared by scripts/verify_cuda.py, alongside small independent exhaustive checks. Benchmark summary equality alone is not a substitute for full pattern verification.

2. Manual validation and reproducibility

The five-basket toy example contains bread, milk, diaper, beer and eggs. With absolute minimum support 3, eight frequent itemsets are expected and were confirmed on the CPU reference: bread(4), milk(4), diaper(4), beer(3), bread+milk(3), bread+diaper(3), milk+diaper(3), and beer+diaper(3).

Additional CPU tests use 100 randomized small datasets checked against an independent subset-enumeration oracle. The CUDA host-simulator code path was tested separately, but a simulator is not proof of actual NVIDIA kernel execution. When performing lab benchmarks, run the strict itemset/support verifier before timing, and retain its terminal log.

2.1 Experiment design

Each of the ten input files is evaluated at four decreasing support thresholds. The thresholds are intentionally dataset-specific to obtain feasible yet informative pattern counts. The program records execution time including input parsing and mining, FP-tree metrics, patterns, maximum pattern length, process peak resident memory, and CUDA buffer high-water when available. Higher pattern counts generally increase work, but memory and runtime also depend on transaction length, unique items, tree compression and number of conditional-tree builds.

2.2 Avoiding misleading comparisons

CPU and CUDA times are compared only when measured on the same machine in the same script run; the plotted ratio is CPU median time / CUDA median time. A value greater than 1 means CUDA was faster, a value less than 1 means CPU was faster. GPU device buffer high-water excludes driver and context allocations and is NOT directly equivalent to peak GPU process memory. The program timer starts before input parsing and ends after mining; it excludes Python startup and bulk CSV pattern writing (--no-output), but includes CUDA launch/transfer overhead as measured inside the C++ binary.

Dataset selection: UCI Iris, UCI Wine, UCI Wisconsin Diagnostic Breast Cancer, UCI Handwritten Digits (all discretized as documented), plus synthetic Grocery, Bundles, Clickstream, Sensors, Long-tail, and Retail 50k. The six synthetic files are for stress testing and are not described as UCI benchmarks.

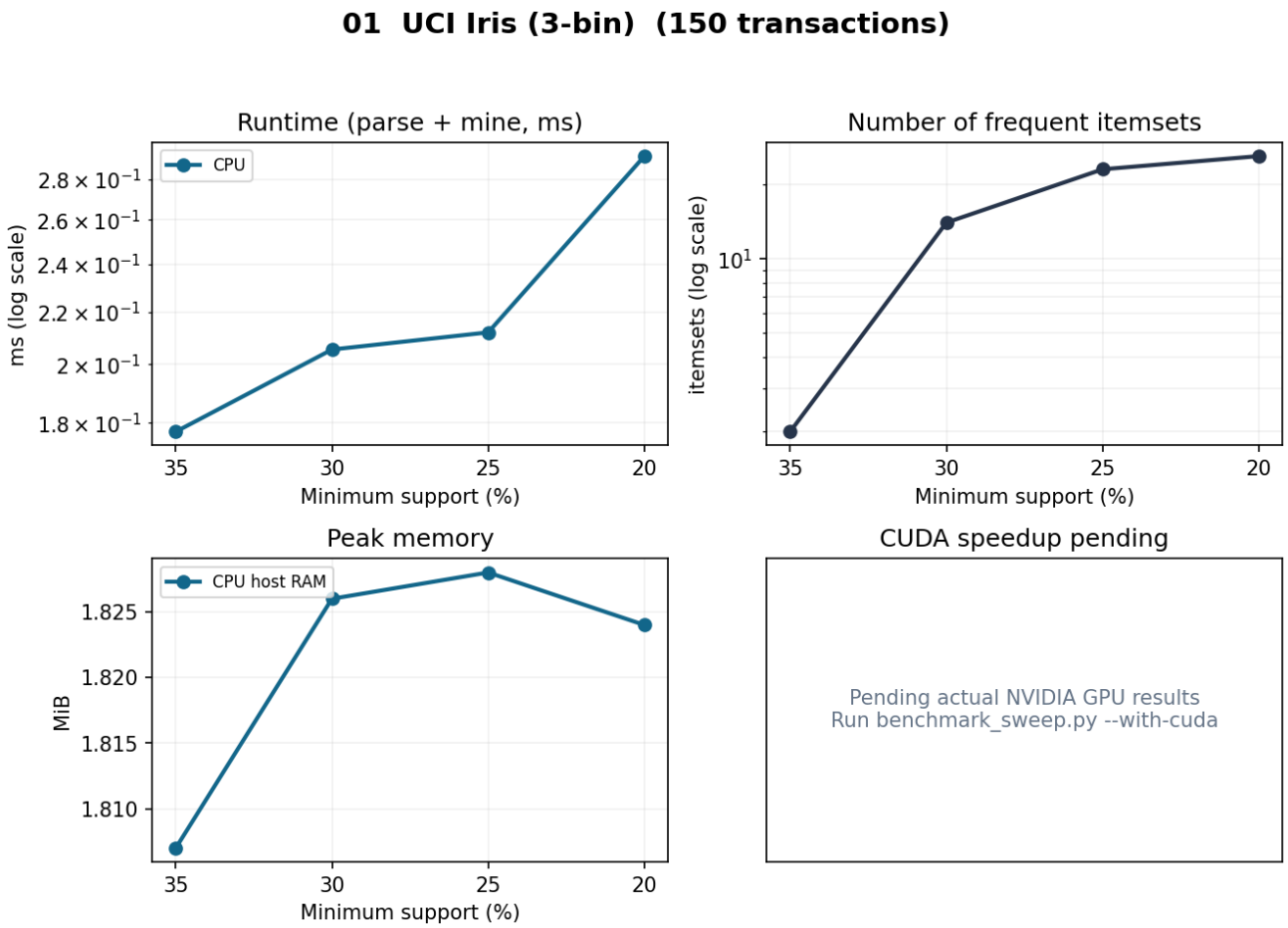
3. The ten datasets and experimental settings

# / dataset	Origin	Rows	Support sweep	Patterns at lowest
1. Iris	UCI-derived	150	35% to 20%	26
2. Wine	UCI-derived	178	50% to 30%	153
3. WDBC	UCI-derived	569	50% to 35%	1,684
4. Digits	UCI-derived	1797	90% to 75%	1,228
5. Grocery	Synthetic	3000	20% to 10%	143
6. Bundles	Synthetic	6000	20% to 12%	60
7. Clickstream	Synthetic	12000	4% to 1.5%	71
8. Sensors	Synthetic	8000	45% to 30%	32
9. Long-tail	Synthetic	20000	10% to 4%	127
10. Retail 50k	Synthetic	50000	10% to 4%	164

Every file above is part of the project ZIP. Data origins, transformations and reproducibility notes are provided in data/TEN_DATASET_SOURCES.md. The support settings are fixed in scripts/benchmark_sweep.py so repeating the experiment does not require editing the report.

What is shown next: each dataset receives a separate page with CPU runtime and CUDA runtime (when measured), number of frequent itemsets, memory, and speedup against minimum support; a table provides the exact measurements behind every plotted point.

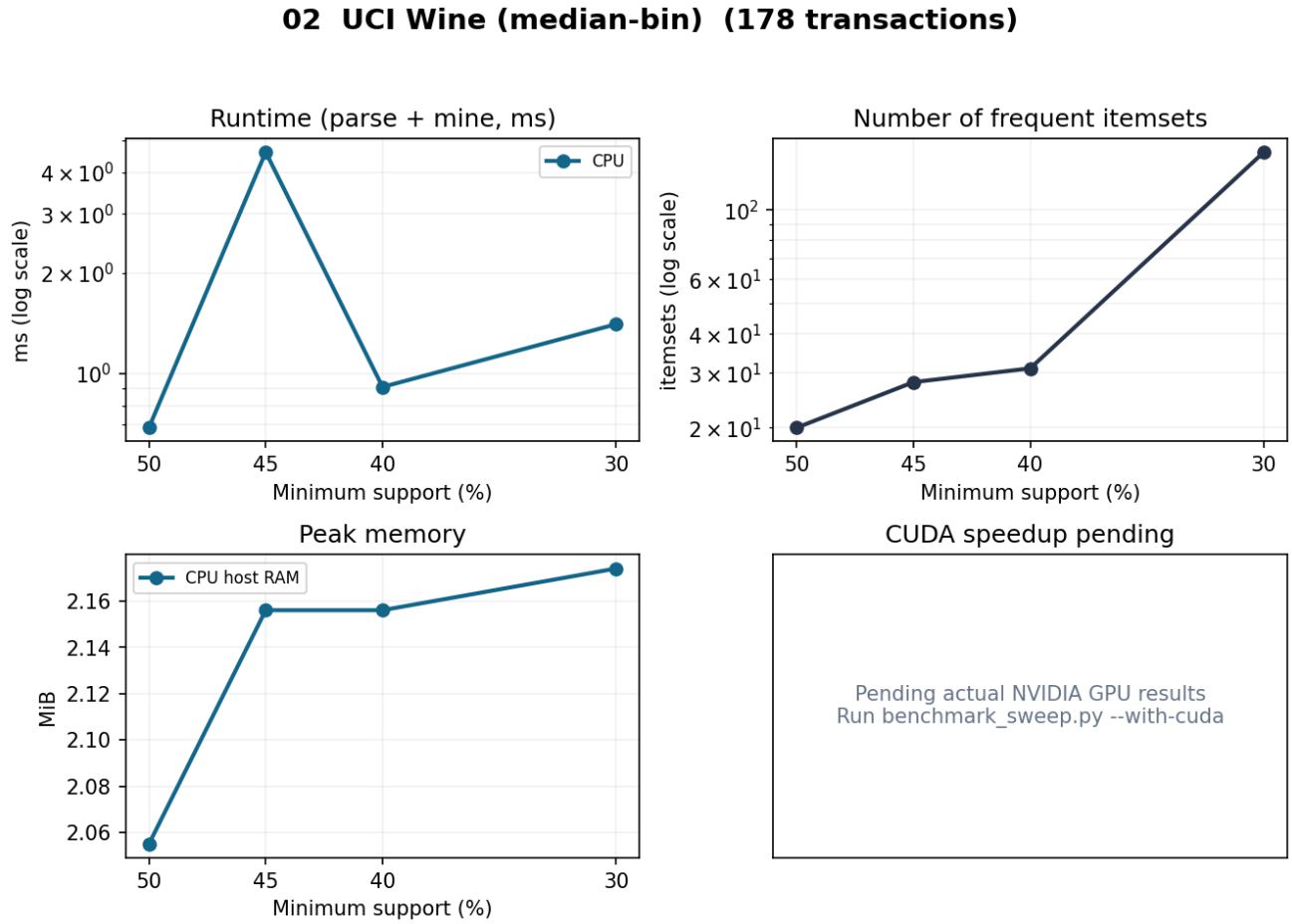
4. Dataset analysis: UCI Iris (3-bin)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
35%	2	0.177	Pending	1.81	Pending
30%	14	0.205	Pending	1.83	Pending
25%	23	0.212	Pending	1.83	Pending
20%	26	0.292	Pending	1.82	Pending

Lowering support from 35% to 20% changed the frequent itemset count from 2 to 26. At the lowest support, the CPU median runtime was 0.29 ms and peak host RSS was 1.82 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

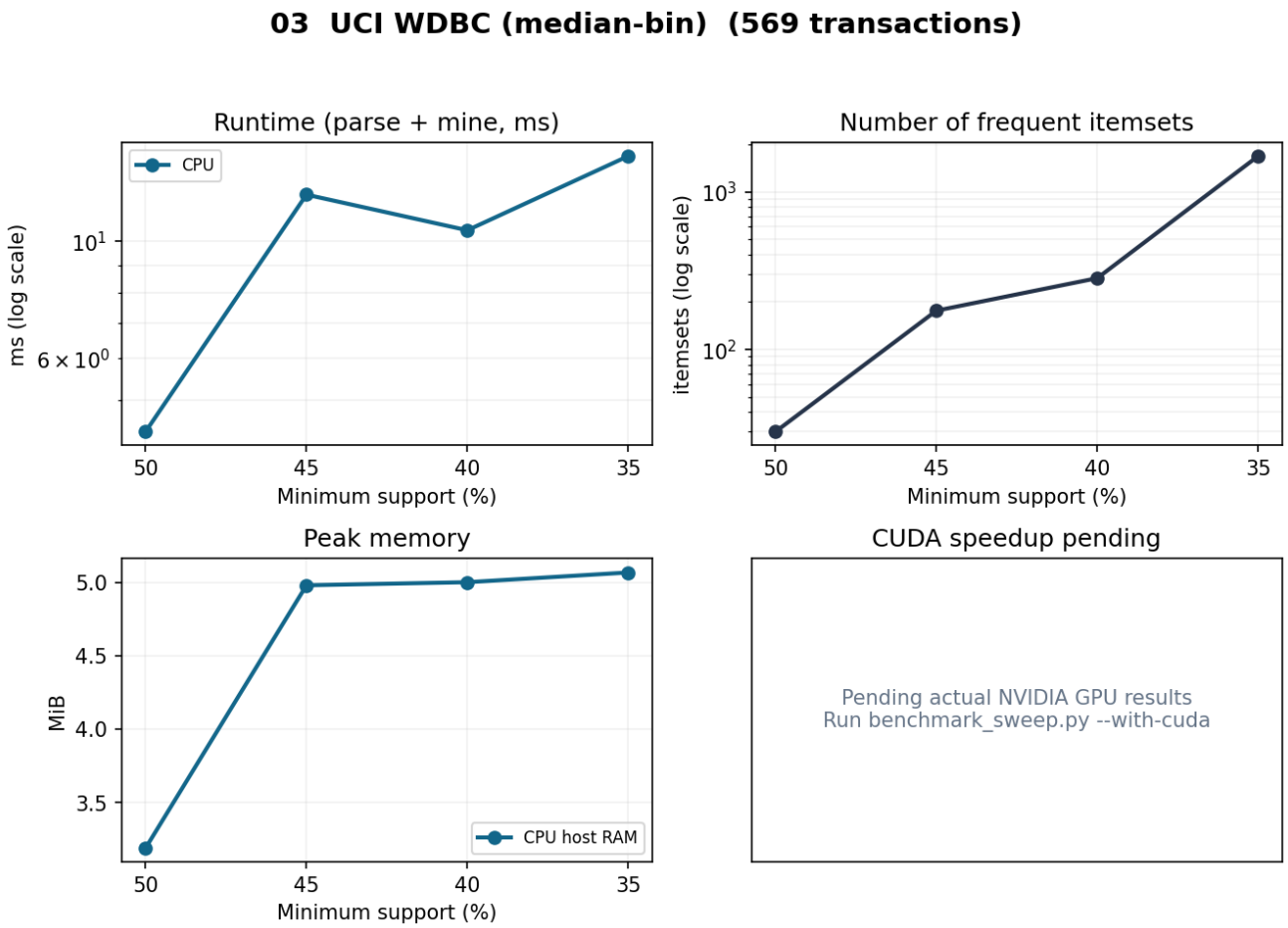
5. Dataset analysis: UCI Wine (median-bin)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
50%	20	0.686	Pending	2.06	Pending
45%	28	4.598	Pending	2.16	Pending
40%	31	0.909	Pending	2.16	Pending
30%	153	1.403	Pending	2.17	Pending

Lowering support from 50% to 30% changed the frequent itemset count from 20 to 153. At the lowest support, the CPU median runtime was 1.40 ms and peak host RSS was 2.17 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

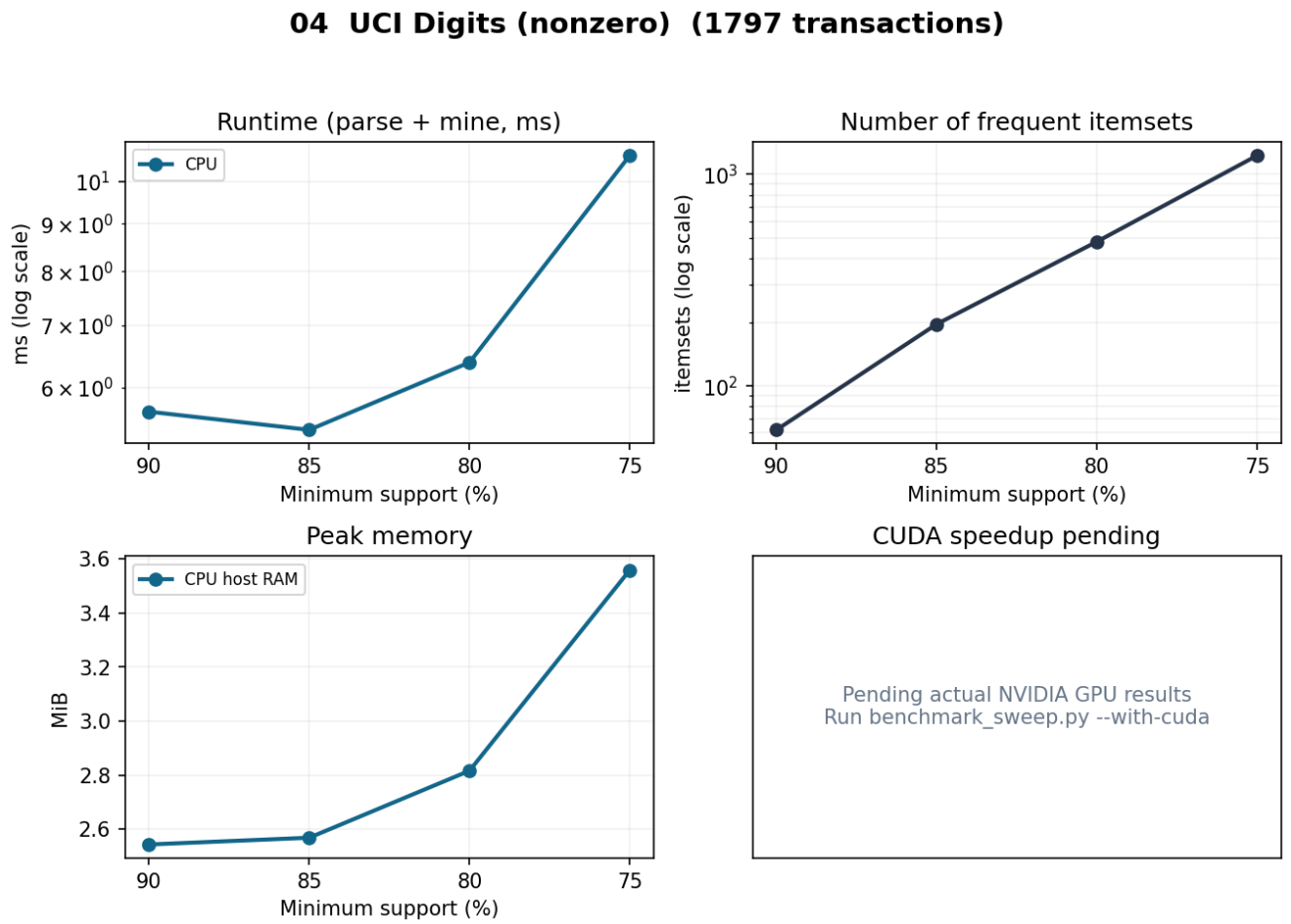
6. Dataset analysis: UCI WDBC (median-bin)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
50%	30	4.350	Pending	3.19	Pending
45%	176	12.261	Pending	4.98	Pending
40%	283	10.492	Pending	5.00	Pending
35%	1,684	14.509	Pending	5.07	Pending

Lowering support from 50% to 35% changed the frequent itemset count from 30 to 1,684. At the lowest support, the CPU median runtime was 14.51 ms and peak host RSS was 5.07 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

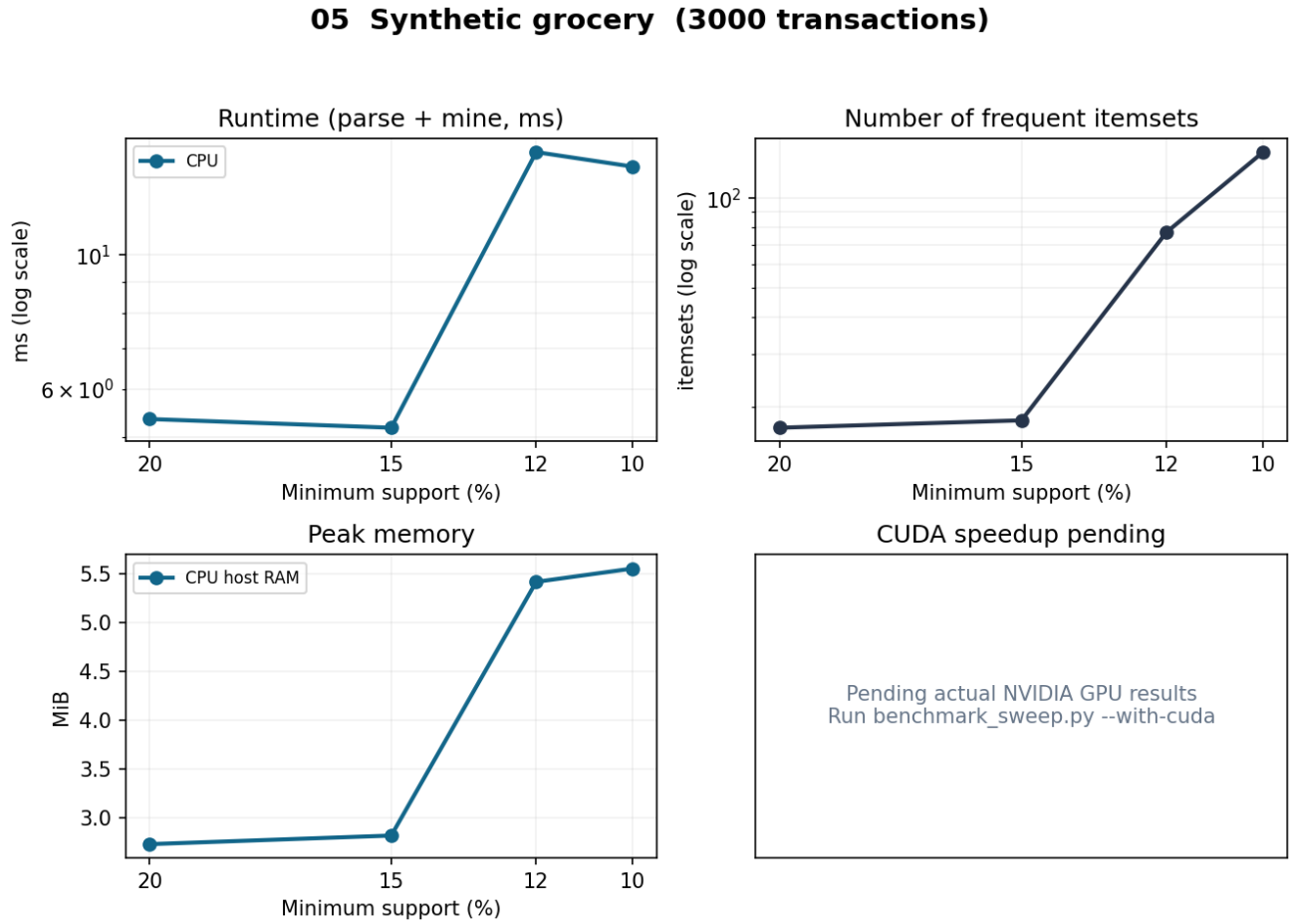
7. Dataset analysis: UCI Digits (nonzero)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
90%	62	5.647	Pending	2.54	Pending
85%	196	5.399	Pending	2.57	Pending
80%	481	6.383	Pending	2.82	Pending
75%	1,228	10.665	Pending	3.56	Pending

Lowering support from 90% to 75% changed the frequent itemset count from 62 to 1,228. At the lowest support, the CPU median runtime was 10.66 ms and peak host RSS was 3.56 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

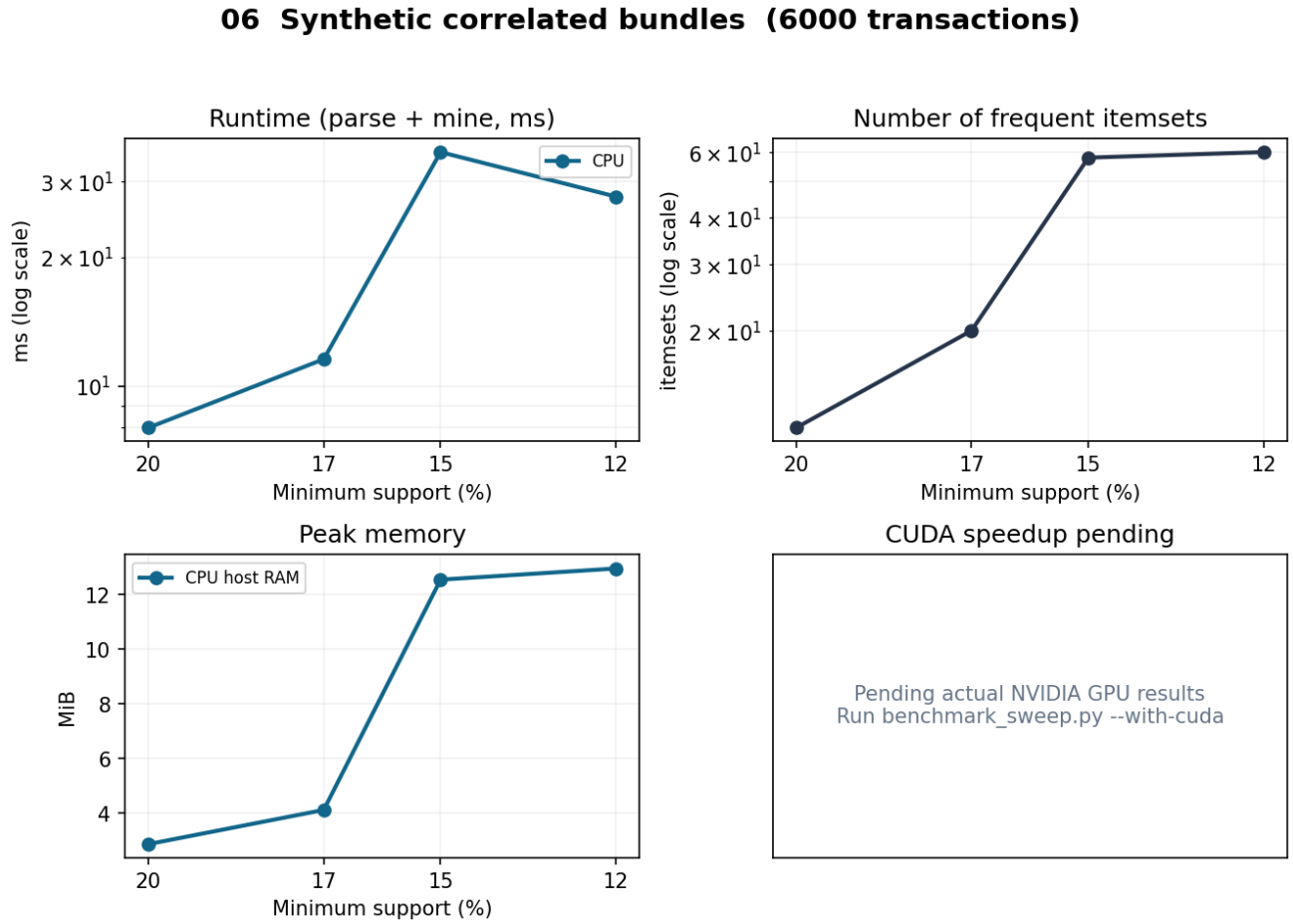
8. Dataset analysis: Synthetic grocery



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
20%	17	5.354	Pending	2.73	Pending
15%	18	5.179	Pending	2.82	Pending
12%	77	14.765	Pending	5.42	Pending
10%	143	13.972	Pending	5.55	Pending

Lowering support from 20% to 10% changed the frequent itemset count from 17 to 143. At the lowest support, the CPU median runtime was 13.97 ms and peak host RSS was 5.55 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

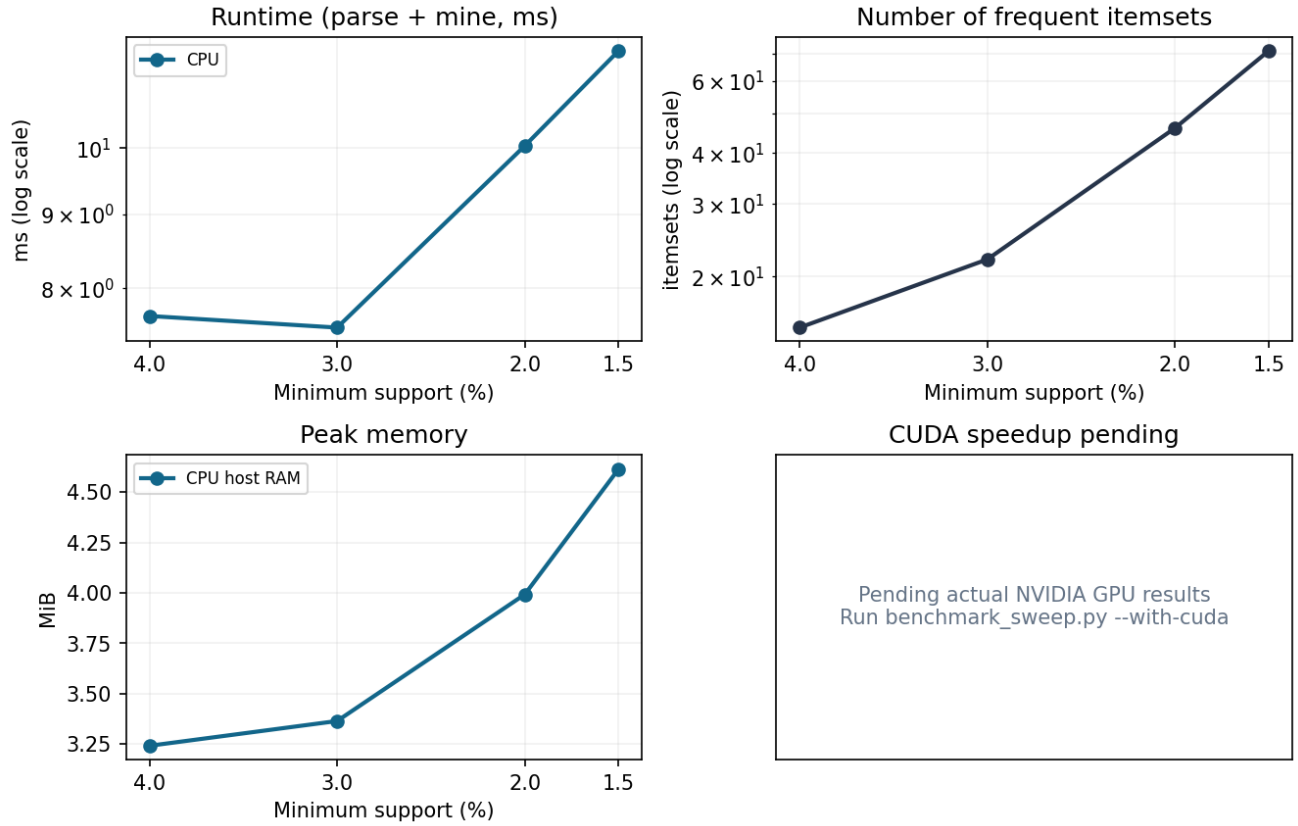
9. Dataset analysis: Synthetic correlated bundles



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
20%	11	7.991	Pending	2.88	Pending
17%	20	11.557	Pending	4.12	Pending
15%	58	35.210	Pending	12.55	Pending
12%	60	27.716	Pending	12.96	Pending

Lowering support from 20% to 12% changed the frequent itemset count from 11 to 60. At the lowest support, the CPU median runtime was 27.72 ms and peak host RSS was 12.96 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

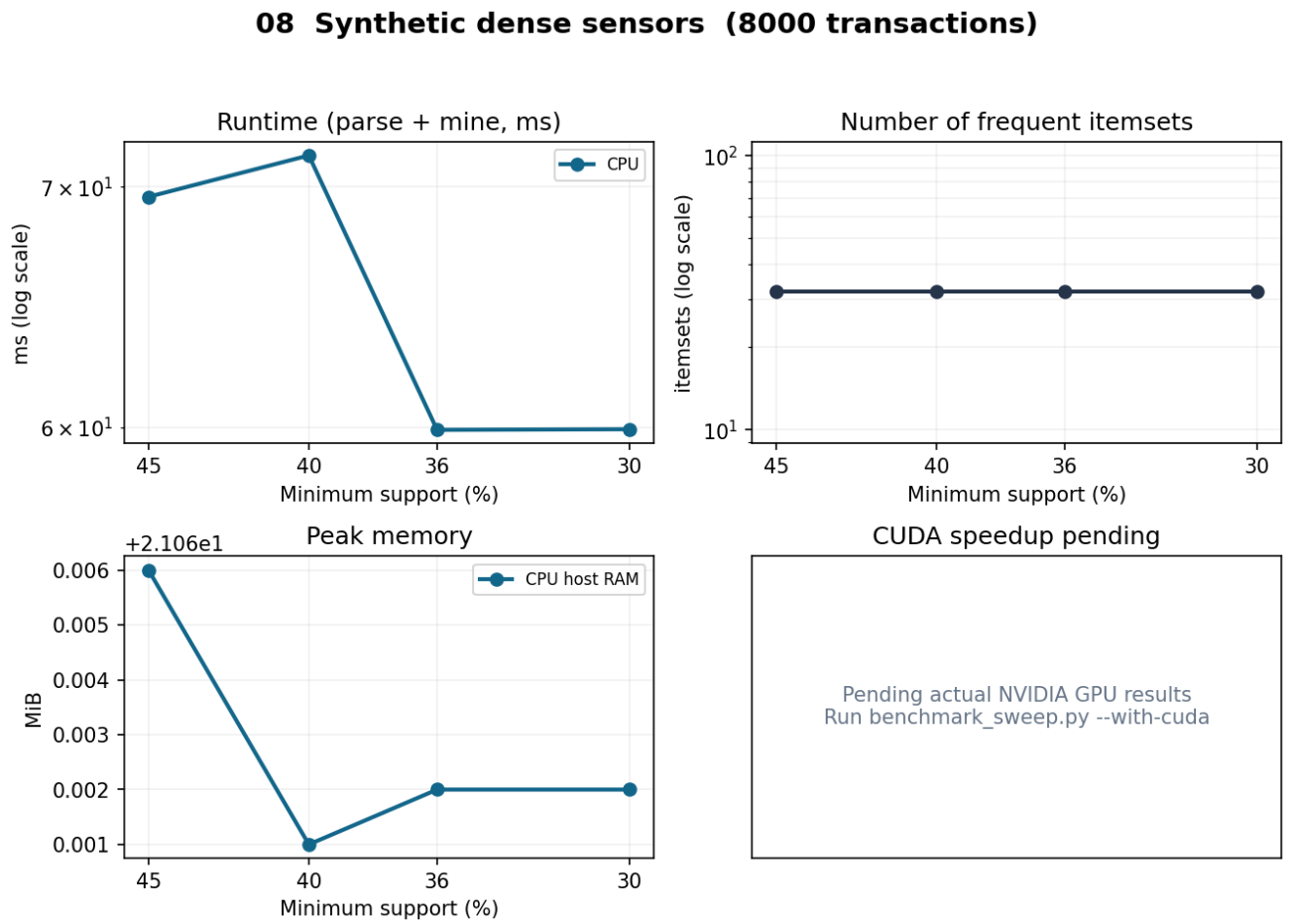
10. Dataset analysis: Synthetic sparse clickstream



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
4%	15	7.655	Pending	3.24	Pending
3%	22	7.516	Pending	3.36	Pending
2%	46	10.035	Pending	3.99	Pending
1.5%	71	11.672	Pending	4.62	Pending

Lowering support from 4% to 1.5% changed the frequent itemset count from 15 to 71. At the lowest support, the CPU median runtime was 11.67 ms and peak host RSS was 4.62 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

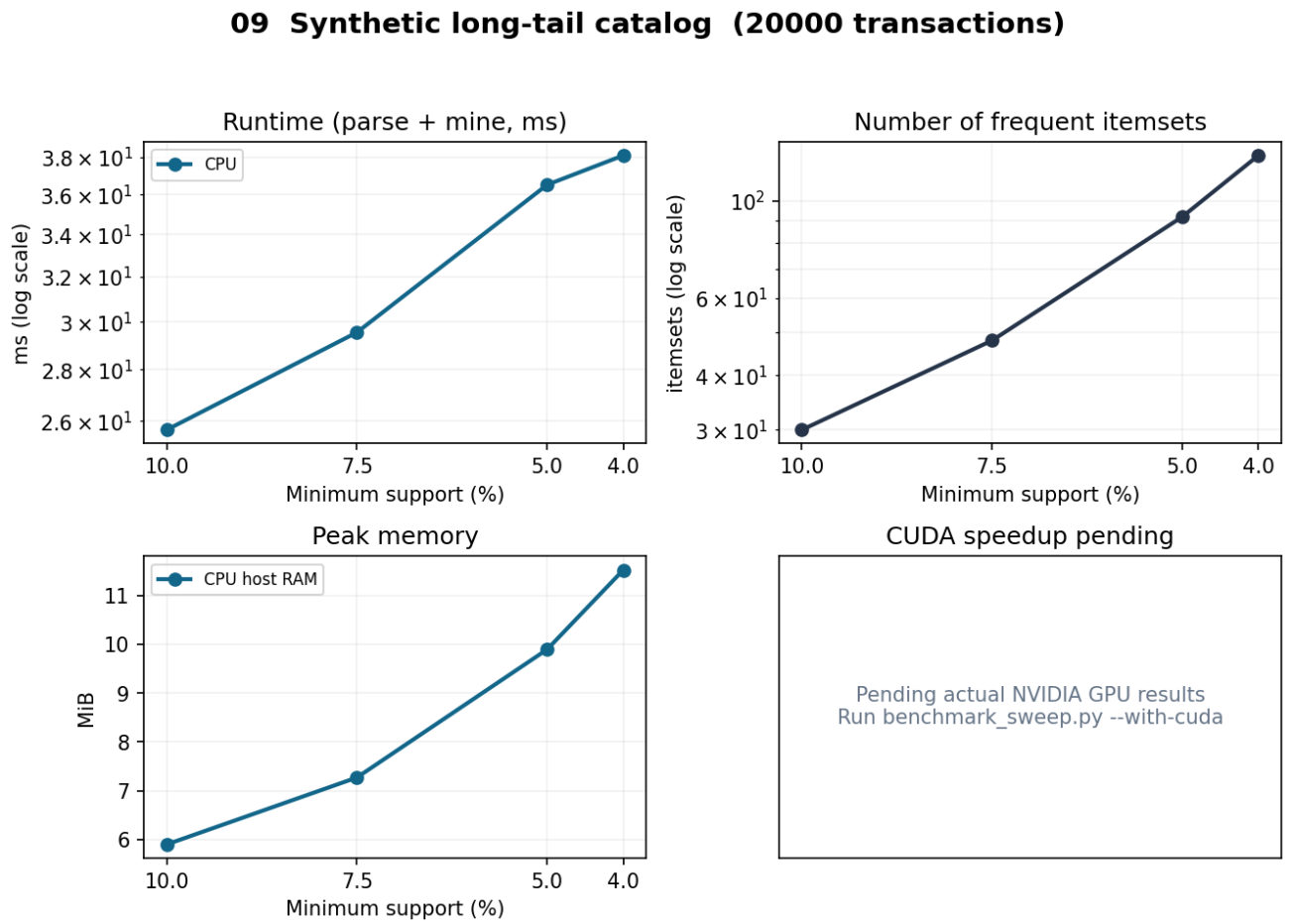
11. Dataset analysis: Synthetic dense sensors



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
45%	32	69.543	Pending	21.07	Pending
40%	32	71.412	Pending	21.06	Pending
36%	32	59.900	Pending	21.06	Pending
30%	32	59.924	Pending	21.06	Pending

Lowering support from 45% to 30% changed the frequent itemset count from 32 to 32. At the lowest support, the CPU median runtime was 59.92 ms and peak host RSS was 21.06 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

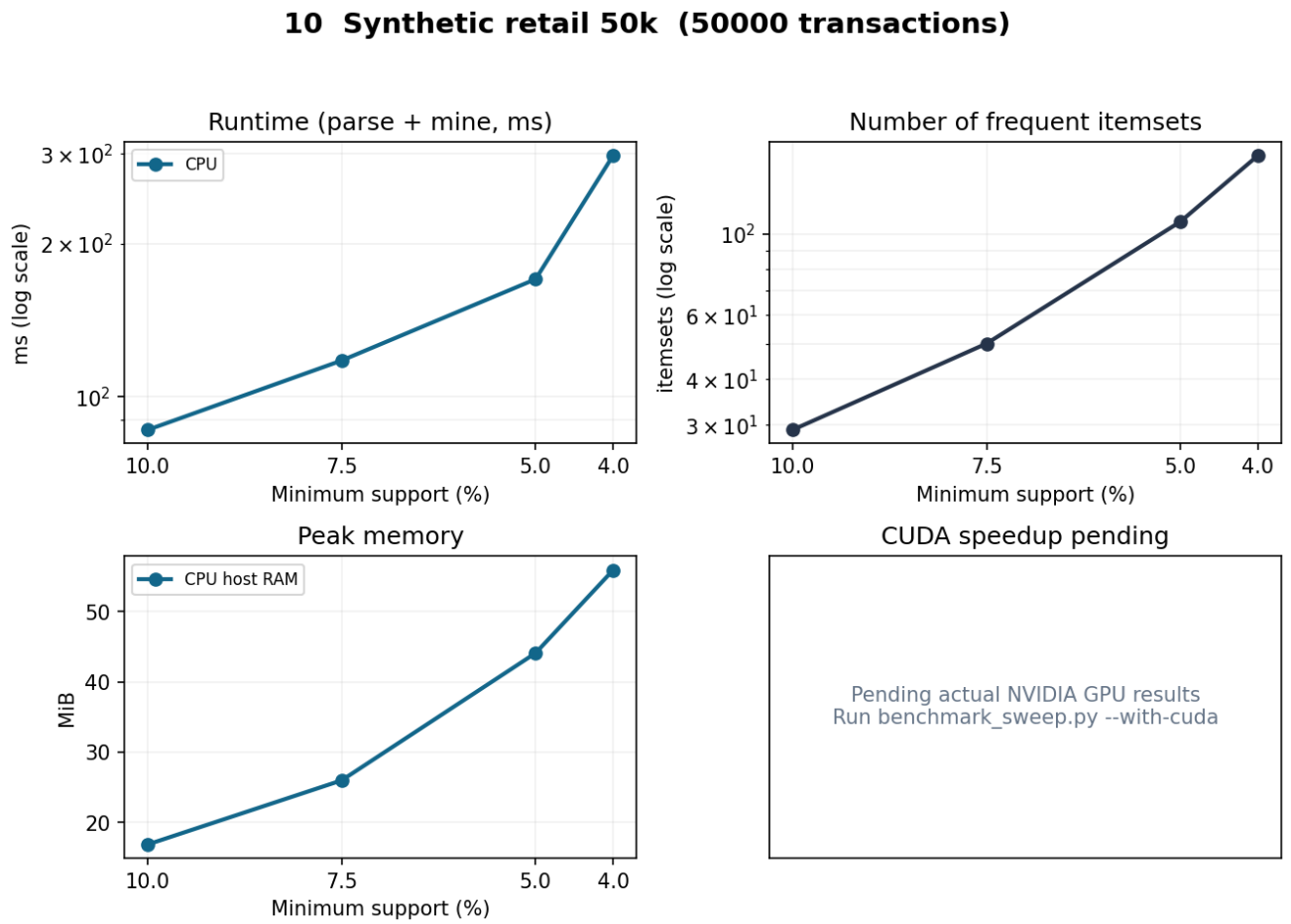
12. Dataset analysis: Synthetic long-tail catalog



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
10%	30	25.677	Pending	5.90	Pending
7.5%	48	29.537	Pending	7.27	Pending
5%	92	36.514	Pending	9.90	Pending
4%	127	38.087	Pending	11.52	Pending

Lowering support from 10% to 4% changed the frequent itemset count from 30 to 127. At the lowest support, the CPU median runtime was 38.09 ms and peak host RSS was 11.52 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

13. Dataset analysis: Synthetic retail 50k



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
10%	29	86.095	Pending	16.85	Pending
7.5%	50	117.807	Pending	25.98	Pending
5%	108	170.175	Pending	44.10	Pending
4%	164	297.787	Pending	55.93	Pending

Lowering support from 10% to 4% changed the frequent itemset count from 29 to 164. At the lowest support, the CPU median runtime was 297.79 ms and peak host RSS was 55.93 MiB. NVIDIA CUDA benchmarks are pending; no GPU performance claim is made.

14. Discussion and conclusions

On the prepared datasets, decreasing the support threshold increases or preserves the number of frequent itemsets, as required mathematically. Runtime does not have to increase monotonically because of measurement noise and differences in tree traversal, but for most datasets a lower threshold makes more itemsets and potentially more conditional trees. The sensitivity to support is why a single support setting cannot characterize FP-Growth performance.

This implementation retains FP-tree construction, recursive traversal and conditional-base management on the CPU, so GPU acceleration is confined to repeated weighted histograms. It differs materially from a GPU-resident FP-Growth design. Many small CUDA launches can dominate end-to-end execution even if histogram operations parallelize well. Conclusions about GPU speedup should be based on the 40 actual same-machine measurements, not performance figures from another implementation or GPU.

Limitations and future work

The prepared ten-dataset bundle comprises four UCI-derived files and six synthetic workloads. A requirement for ten original UCI datasets would need six replacements before submission. The chosen thresholds are not uniform and were selected for feasible runtime, so cross-dataset timing should be interpreted within each dataset rather than as a controlled comparison of difficulty. Small datasets have timing noise. A fully GPU-resident implementation or conditional-pattern-base batching could change the performance balance, but neither is claimed as implemented.

References

Han, J., Pei, J. and Yin, Y. (2000). Mining Frequent Patterns without Candidate Generation. ACM SIGMOD. https://doi.org/10.1145/342009.335372

UCI Machine Learning Repository. https://archive.ics.uci.edu/

NVIDIA CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/

15. Lab commands and submission checklist

```
make
make cuda
python3 scripts/verify_cuda.py --random-cases 20
python3 scripts/benchmark_sweep.py --with-cuda --repeats 3
python3 scripts/generate_sweep_report.py
```

Install PDF dependencies through Ubuntu system packages if they are not available: sudo apt install python3-reportlab python3-matplotlib. Do not use sudo pip, especially on a shared lab workstation. Alternatively create a Python virtual environment if permitted.

Save the strict comparison output as a log, verify that the report explicitly names the NVIDIA GPU and prints CPU/GPU speedup for all forty settings, inspect all ten charts, and only then print the PDF and email your own source code, datasets and report in the ZIP. Ensure the assignment due date is confirmed with the instructor.

Source note: This report is generated from results/support_sweep_results.csv created by this project. The comparison project supplied by a classmate was used to identify useful experimental presentation practices, not as a source for benchmark numbers or as code to copy.